오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 기초

DAY 10

JavaScript 개요와 기본 문법

정적인 페이지에 동작을 더하는 첫날입니다.
브라우저 콘솔에서 변수와 자료형을 직접 확인합니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **개발자 도구 콘솔에서 JavaScript를 실행할 수 있다**

   Console에 바로 치고, script 태그로 연결한 파일의 결과를 봅니다.

2. **var · let · const로 값을 담고 바꿀 수 있다**

   다시 담을 수 있는 것과 없는 것을 구분하고, 이름을 규칙대로 짓습니다.

3. **자료형을 typeof로 확인하고 오류를 찾아 고칠 수 있다**

   다섯 가지 기본 자료형과 세 가지 오류 메시지를 읽습니다.

오늘의 결과물

콘솔에 내 정보를 자료형과 함께 출력하는 profile.js — 일부러 낸 오류 세 개까지 고친 상태

ROADMAP

오늘의 흐름

01

JavaScript의 역할

HTML·CSS와 나누는 일
역사와 표준

02

개발자 도구와 콘솔

Console에서 실행
파일 연결과 출력

03

변수

var · let · const
실습 01

실습 포함

04

자료형과 디버깅

typeof · 오류 읽기
실습 02

실습 포함

앞의 두 섹션은 JavaScript가 무엇이고 어디서 실행하는지, 뒤의 두 섹션은 값을 담고 확인하는 법입니다.
오늘은 화면을 바꾸지 않고 모든 결과를 콘솔에서 봅니다.

되짚기 · 01

지금까지는 화면을 만들었습니다

```
<!-- DAY 01~09에서 만든 것 -->
<header> … </header>          ← HTML : 무엇이 있는지
.card { padding: 20px; }      ← CSS : 어떻게 보이는지
@media (min-width: 768px)     ← 화면 폭에 따라 다르게
<div class="col-md-6">        ← Bootstrap : 빠르게 배치

<script src="…bootstrap.bundle.min.js">  ← 이것만 JavaScript
```

보여 주는 화면

HTML과 CSS는 적어 둔 글과
모양을 보여 줍니다.
입력에 따라 내용을 바꾸는
일은 하지 못합니다.

오늘부터

누르면 열리고, 입력을 검사하고,
값을 계산하는 코드를
JavaScript로 씁니다.

모달을 움직이기 위해 JavaScript 사용

```
<button class="btn btn-primary" data-bs-toggle="modal"
        data-bs-target="#subscribe">구독하기</button>
…
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8
        /dist/js/bootstrap.bundle.min.js"></script>

/* 이 파일이 JavaScript입니다 — 지우면 모달이 열리지 않았습니다 */
```

남이 쓴 JavaScript

버튼을 누르면 상자를
띄우는 코드를 Bootstrap이
미리 써 두었습니다.

오늘부터는 직접

같은 script 태그로
내가 쓴 .js 파일을
연결하는 것부터 합니다.

준 비

오늘의 셋팅

```
Day10/                    ← 같은 저장소 안에 만듭니다
  index.html              ← main.js 를 연결합니다 (실습 01)
  main.js
  profile.html            ← profile.js 를 연결합니다 (실습 02)
  profile.js
$ git status
/* 오늘도 main에 바로 커밋합니다 */
```

Chrome으로 엽니다

개발자 도구의 메뉴와
단축키가 화면과 같아야
따라오기 쉽습니다.

인터넷은 없어도 됩니다

오늘은 CDN을 쓰지 않습니다.
파일을 더블클릭해 열어도
결과가 똑같이 나옵니다.

01

JavaScript의 역할

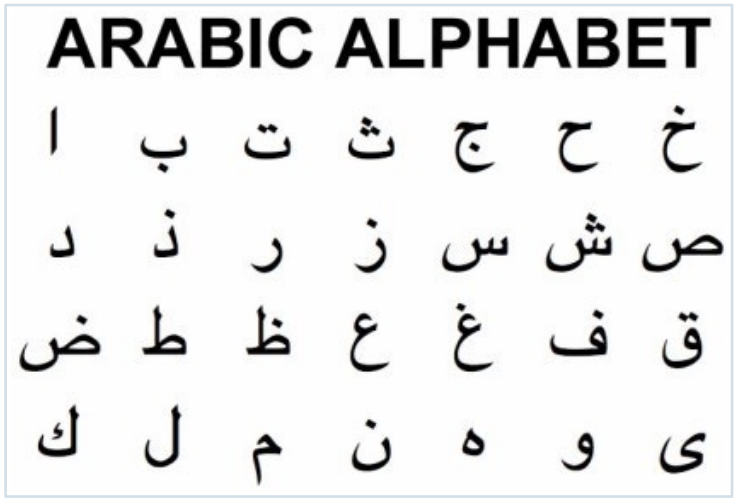
HTML·CSS와 나누는 일 · 실행 방식 · 역사와 표준 · 실행 환경

새 언어를 배우는 첫날입니다

이 문장, 읽을 수 있나요?

مرحبًا وابتهج اليوم!

뜻 : 안녕하세요, 오늘 하루 즐겁게 보내세요!



아랍어 알파벳 28자

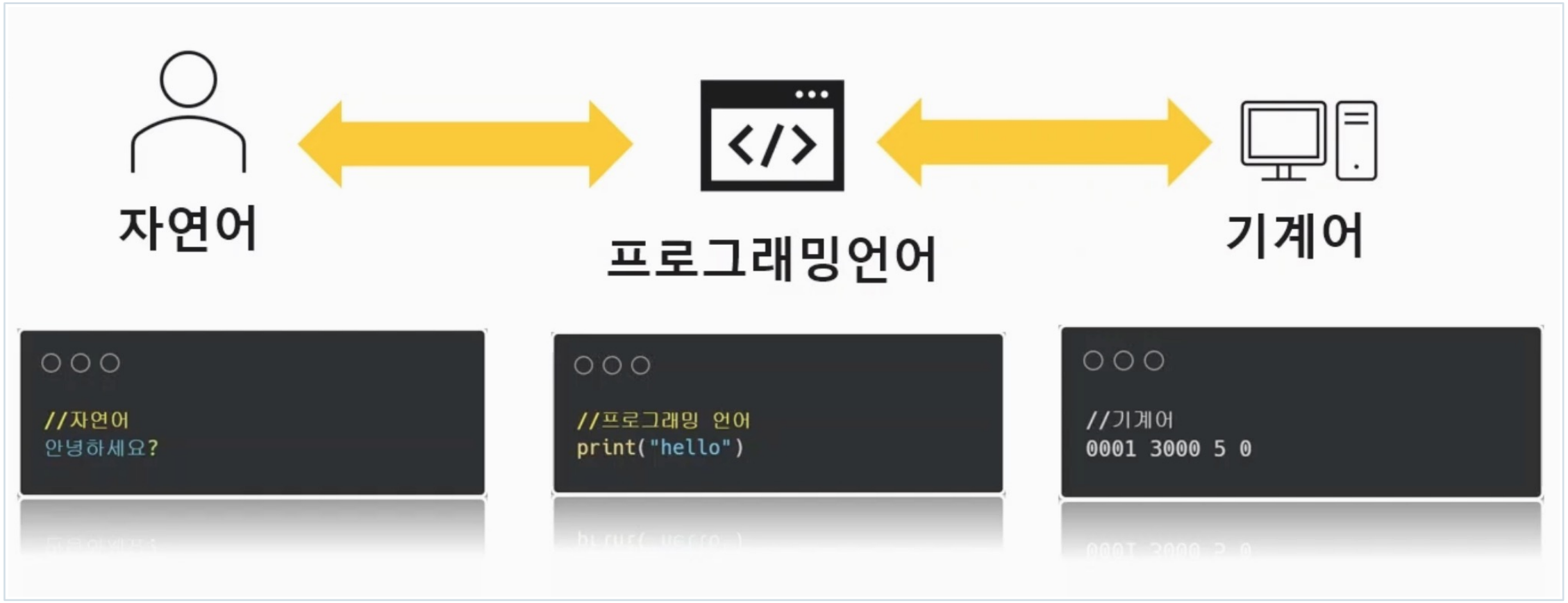
모르는 글자는 못 읽습니다

글자 모양과 읽는 법,
문법을 알아야
문장이 읽힙니다.

JavaScript도 같습니다

오늘은 코드의 첫 글자들,
값을 담고 확인하고
출력하는 법을 익힙니다.

프로그래밍 언어는 사람과 기계 사이의 언어입니다



왼쪽부터 사람이 쓰는 말, 프로그래밍 언어, 기계어 — 아래 창은 각각의 말로 적은 예입니다

01 · 역할

HTML은 구조, CSS는 모양, JavaScript는 동작

01

HTML

무엇이 있는지 적습니다.
제목, 문단, 버튼,
입력칸이 여기에 속합니다.

02

CSS

어떻게 보이는지 정합니다.
색, 크기, 배치와
화면 폭별 모양까지.

03

JavaScript

어떻게 움직이는지 씁니다.
누르면 열고, 입력을 검사하고,
값을 계산합니다.

세 언어는 한 페이지에서 함께 일합니다 — 오늘은 세 번째를 시작합니다

01 · 예시

JavaScript가 하는 일은 이런 것입니다

01

누르면 열린다

DAY 08의 모달과
좁은 화면에서 접히는 메뉴가
JavaScript로 움직였습니다.

02

입력을 검사한다

비밀번호 두 칸이 같은지처럼
HTML만으로 못 하는 검사를
보내기 전에 합니다.

03

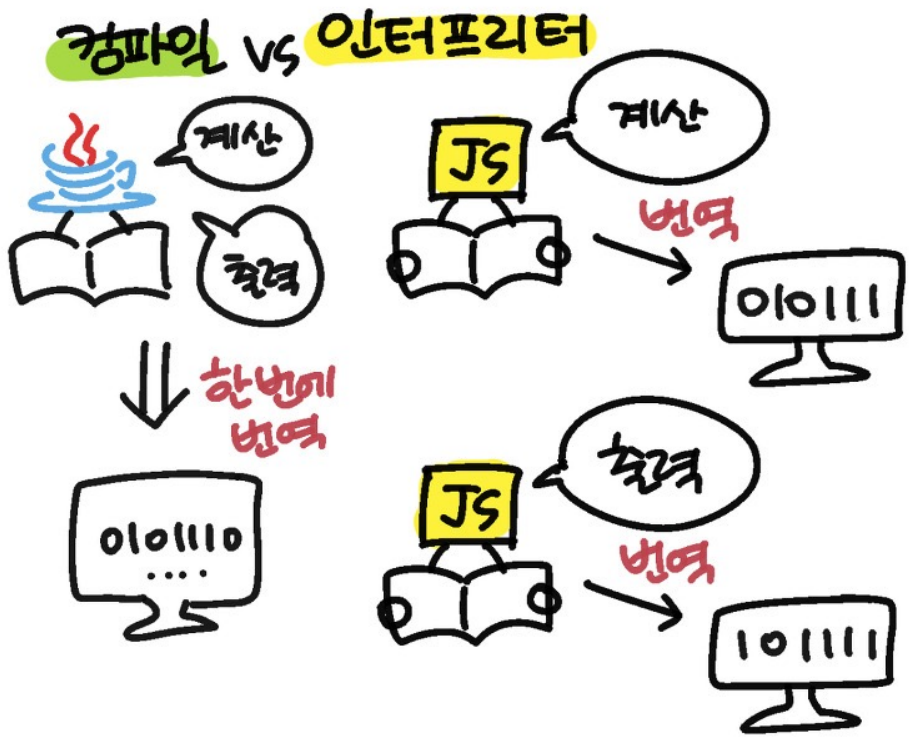
데이터를 가져온다

서버에 있는 목록을 받아
화면에 카드로 그립니다.
새로고침 없이 바뀝니다.

오늘은 그 전 단계 — 값을 담고, 확인하고, 출력하기

화면을 바꾸는 방법은 DAY 15에서 배웁니다. 오늘 결과는 모두 콘솔에서 봅니다.

JavaScript는 받은 코드를 바로 실행합니다



왼쪽은 한 번에 번역해 두고 실행, 오른쪽은 실행하면서 한 줄씩 번역

컴파일 언어

C나 Go는 소스 코드를 먼저
기계어 실행 파일로 바꾸고,
그 파일을 실행합니다.

JavaScript

소스 코드 그대로 브라우저에
전달되고, 브라우저 안의
엔진이 바로 실행합니다.
Chrome의 엔진은 V8입니다.

1995년, 넷스케이프에서 시작했습니다

1	1995	넷스케이프의 브렌던 아이크가 첫 버전을 열흘 만에 만들었습니다
2	1995	Mocha, LiveScript를 거쳐 12월에 JavaScript라는 이름이 붙었습니다
3	1997	ECMAScript라는 이름으로 표준(ECMA-262)이 만들어졌습니다
4	2009	Node.js가 나와 서버에서도 널리 쓰이기 시작했습니다
5	2015	ES2015(ES6)에서 let, const, 백틱 문자열이 들어왔습니다

이름만 비슷할 뿐 Java와는 다른 언어입니다 — 당시 인기 있던 Java의 이름을 빌렸습니다

ECMAScript는 JavaScript의 표준입니다

```
// ES5(2009)까지의 방식
var userName = "김오즈";
var greeting = "안녕하세요, " + userName + "님";

// ES2015(ES6) 이후의 방식
const userName = "김오즈";
const greeting = `안녕하세요, ${userName}님`;
/* 같은 일을 하는 두 시절의 코드입니다 */
```

ES6 = ES2015

2015년 6월에 나온
6번째 판입니다. let, const와
백틱 문자열이 이때
생겼습니다.

검색할 때

var만 가득한 예제는
2015년 이전 방식일
가능성이 큽니다.

브라우저 밖에서도 실행됩니다

01

브라우저

Chrome은 V8, Safari는 JavaScriptCore, Firefox는 SpiderMonkey로 실행합니다.

02

서버

2009년에 나온 Node.js로 서버를 만듭니다. 화면과 서버를 한 언어로 씁니다.

03

데스크톱 앱

지금 쓰는 VS Code도 JavaScript로 앱을 만드는 Electron으로 만들었습니다.

1인 창업가에게는 한 언어로 끝까지 가는 길

이 과정에서도 DAY 27부터 React로 화면을, DAY 49부터 Node.js로 서버를 만듭니다.

여기까지 정리

1	**역할**	HTML은 구조, CSS는 모양, JavaScript는 동작을 맡습니다
2	**실행**	소스 코드 그대로 브라우저에 전달되고, 엔진이 바로 실행합니다
3	**표준**	표준 이름은 ECMAScript, ES2015에서 let과 const가 생겼습니다
4	**환경**	브라우저와 Node.js, 데스크톱 앱까지 같은 언어로 씁니다

다음은 코드를 실행해 볼 자리, 개발자 도구입니다

02

개발자 도구와 콘솔

여는 법 · Console · script 연결 · console 출력

02 · 여는 법

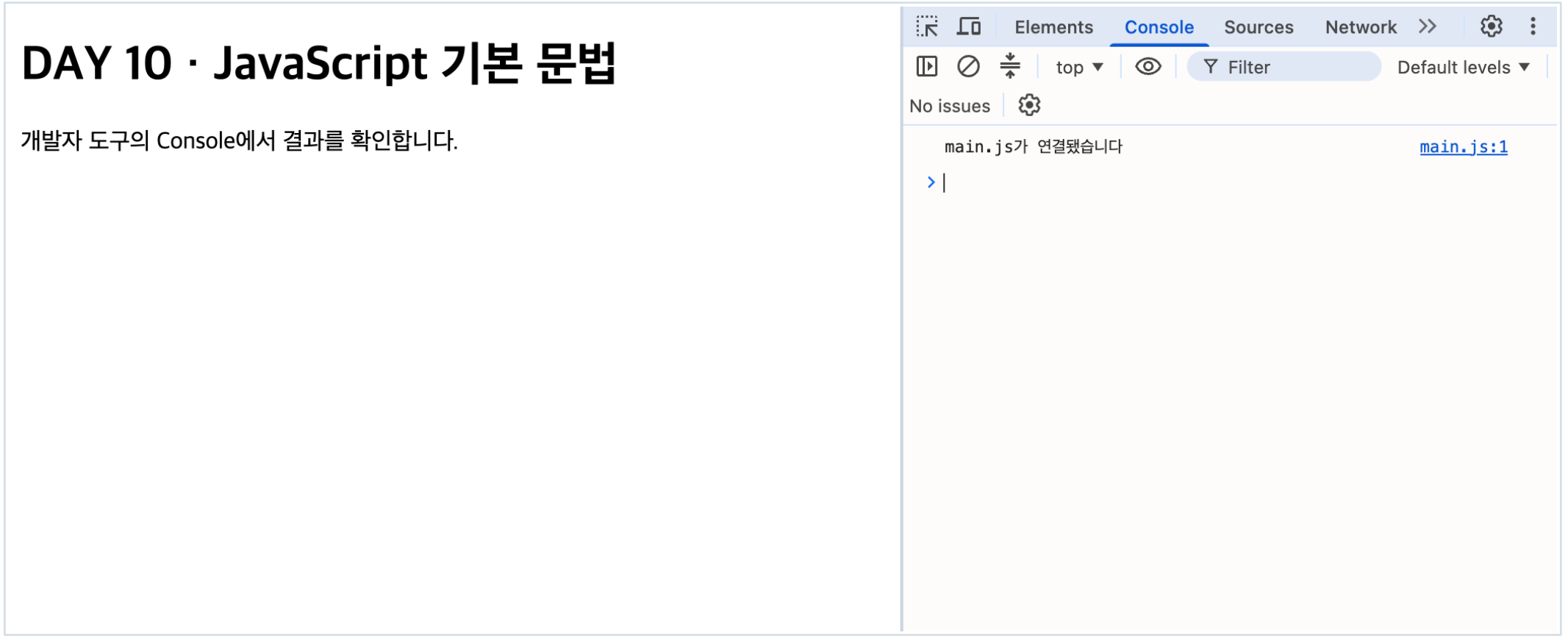
개발자 도구는 이렇게 엽니다

하는 일	Windows	Mac
개발자 도구 열기	F12 또는 Ctrl + Shift + I	Cmd + Option + I
Console 탭 바로 열기	Ctrl + Shift + J	Cmd + Option + J
요소를 골라 검사하기	Ctrl + Shift + C	Cmd + Shift + C
마우스로 열기	페이지에서 우클릭 → 검사	페이지에서 우클릭 → 검사

크롬이 한국어면 개발자 도구도 한국어로 나옵니다

탭 이름이 요소 · 콘솔 · 소스 · 네트워크로 바뀔 뿐 기능은 같습니다.

개발자 도구를 열면 이렇게 보입니다



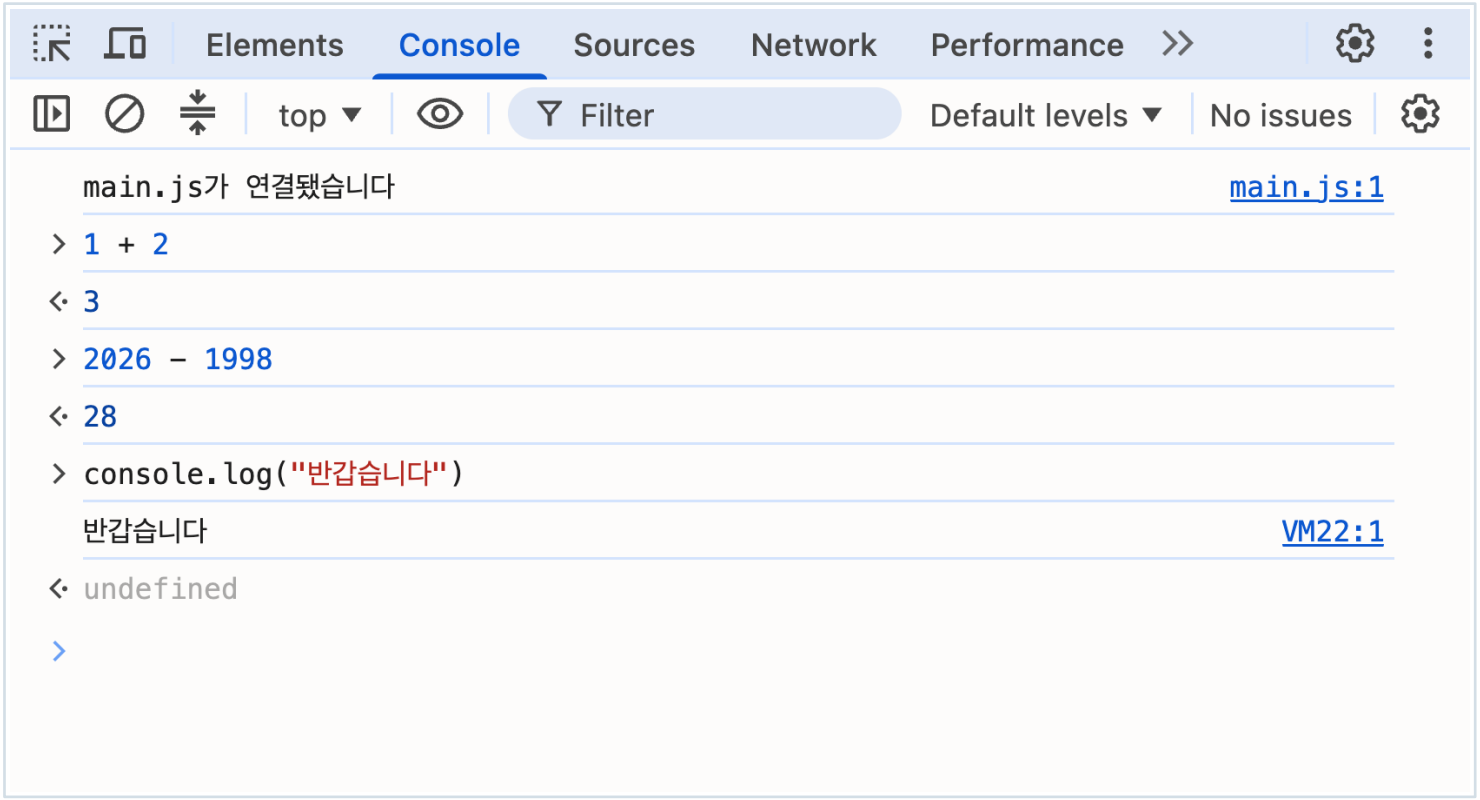
왼쪽이 페이지, 오른쪽이 개발자 도구 — 맨 위 줄에서 Elements · Console · Sources · Network 탭을 고릅니다

자주 쓰는 탭은 네 개입니다

탭	하는 일	오늘 쓰는 곳
Elements (요소)	HTML 구조와 적용된 CSS를 봅니다	DAY 04부터 써 온 탭
Console (콘솔)	코드를 실행하고 출력과 오류를 봅니다	오늘 대부분
Sources (소스)	불러온 파일을 보고 실행을 멈춰 봅니다	디버깅
Network (네트워크)	파일을 제대로 받아 왔는지 봅니다	파일 연결 확인

오늘은 Console이 중심입니다 — 오류가 나면 Sources로 따라갑니다

Console에 바로 쳐서 실행합니다



> 뒤에 치고 Enter

결과가 바로 아래에 나옵니다.
< 뒤의 값은 그 코드가
돌려준 결과입니다.

undefined는 오류가 아닙니다

console.log는 글자를 찍기만
하고 돌려주는 값이 없어서
undefined가 함께 나옵니다.

Console을 편하게 쓰는 네 가지

1	**여러 줄**	Shift + Enter로 줄을 바꿉니다. 그냥 Enter는 바로 실행합니다
2	**이전 입력**	↑ 키를 누르면 방금 친 코드가 다시 나옵니다
3	**지우기**	Ctrl + L (Mac은 Cmd + K) 또는 console.clear()
4	**미리보기**	치는 동안 식의 결과가 아래에 회색 글씨로 미리 보입니다 (Eager evaluation)

Console에서 만든 변수는 새로고침하면 사라집니다

남겨야 하는 코드는 .js 파일에 쓰고 script 태그로 연결합니다.

script 태그로 파일을 연결합니다

```
<!-- index.html -->
<body>
  <h1>DAY 10 · JavaScript 기본 문법</h1>
  <p>개발자 도구의 Console에서 결과를 확인합니다.</p>

  <script src="main.js"></script>
</body>

// main.js
console.log("main.js가 연결됐습니다");
```

body 끝에 둡니다

HTML을 다 읽은 뒤
스크립트가 실행됩니다.
DAY 08의 Bootstrap JS와
같은 자리입니다.

연결 확인

새로고침했을 때 문장이 찍히면
연결된 것입니다. 파일을
못 찾으면 빨간 줄이 뜹니다.

코드는 위에서 아래로 한 줄씩 실행됩니다

```
// main.js
console.log("1. 첫 번째 줄");
console.log("2. 두 번째 줄");
// console.log("주석은 실행되지 않습니다");
console.log("3. 세 번째 줄");
/* 여러 줄 주석은
   이렇게 감쌉니다 */
```

Console

```
1. 첫 번째 줄
2. 두 번째 줄
3. 세 번째 줄
```

주석은 // 와 /* */

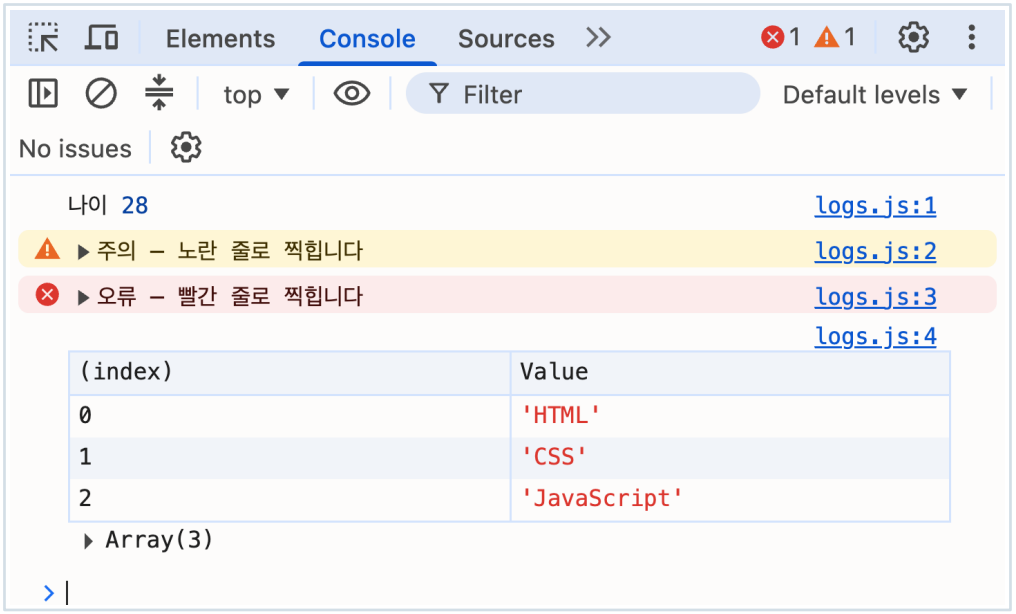
실행되지 않는 메모입니다.
VS Code에서 Ctrl + /
(Mac은 Cmd + /)로 켜고 끕니다.

문장 끝의 세미콜론

; 는 문장이 끝났다는 표시.
빠뜨려도 대부분 자동으로
넣어 주지만 이 과정에서는
붙여 씁니다.

console에는 log 외의 여러 출력이 있습니다

```
console.log("나이", 28);
console.warn("주의 — 노란 줄로 찍힙니다");
console.error("오류 — 빨간 줄로 찍힙니다");
console.table(["HTML", "CSS", "JavaScript"]);  // 표로 보여 줍니다
```



쉼표로 여러 개

console.log("나이", 28)처럼
쉼표로 이으면 사이에 한 칸을
띄워 함께 찍습니다.

종류별로 걸러 보기

warn은 노란 줄, error는 빨간 줄.
Console 위쪽 메뉴에서
종류별로 골라 볼 수 있습니다.

03

변수

선언과 할당 · var · let · const · 이름 짓기 · 실습 01

변수는 값에 붙이는 이름입니다



상자에 이름표(변수 이름)를 붙이고
내용물(값)을 담는 그림입니다.

```
let price = 200;      // price라는 이름에 200을 담습니다
console.log(price);   // 200

price = 300;          // 같은 이름에 300을 다시 담습니다
console.log(price);   // 300
```

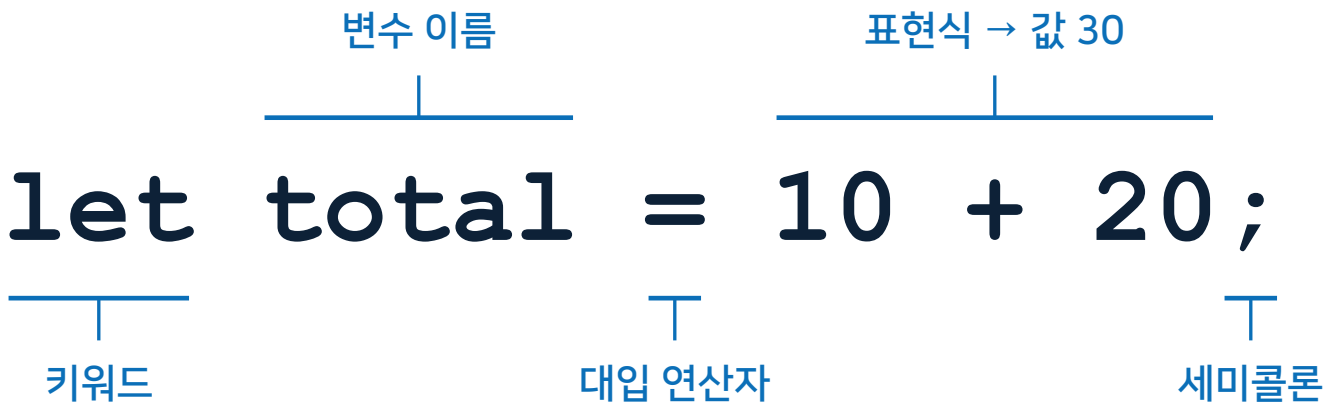
이름으로 꺼내 씁니다

200을 매번 적지 않고
price라는 이름으로 씁니다.
값이 바뀌어도 코드는
그대로입니다.

= 는 「같다」가 아닙니다

오른쪽 값을 왼쪽 이름에
담으라는 뜻입니다. 같은지
비교하는 기호는 DAY 11에서.

한 줄을 나눠 읽어 봅니다



= 오른쪽을 먼저 계산하고, 그 값을 왼쪽 이름에 담습니다

10 + 20을 먼저 계산해 30을 만들고, 그 30이 total에 들어갑니다.

선언하고, 할당하고, 초기값을 넣습니다

```
let score;              // 선언 : 이름을 만듭니다
console.log(score);   // undefined ← 아직 값이 없습니다
score = 90;             // 할당 : 값을 넣습니다
console.log(score);   // 90

let level = 1;          // 초기화 : 선언하면서 처음 값을 넣습니다
```

선언만 하면 undefined

값을 넣지 않은 변수에는
JavaScript가 undefined를
넣어 둡니다.

선언은 한 번만

이미 선언한 score에 다시
넣을 때는 let 없이
score = 값 으로 씁니다.

그릇의 종류는 정하지 않습니다



바구니 — 물을 담을 수 없습니다



컵 — 물을 담을 수 있습니다

```
int cake = 200;      // Java : 담을 종류(int)를 먼저 적습니다
let cake = 200;      // JavaScript : 종류를 적지 않습니다
```

종류를 먼저 정하는 언어

Java는 int score처럼
담을 종류를 먼저 적고,
그 종류의 값만 담습니다.

JavaScript는

let과 const에 종류를
적지 않습니다. 담긴 값을
보고 자료형이 정해집니다.

키워드는 이미 뜻이 정해진 단어입니다

break	case	catch	class	**const**	continue
debugger	default	delete	do	else	export
extends	**false**	finally	for	function	if
import	in	instanceof	new	**null**	return
super	switch	this	throw	**true**	try
typeof	**var**	void	while	with	

언제나 예약어인 35개 (MDN 기준) — 파란 칸은 오늘 쓰는 단어

```
const if = 1;
// SyntaxError: Unexpected token 'if'
let class = "A반";
// SyntaxError: Unexpected token 'class'
```

예약어

언어가 쓰려고 맡아 둔 단어입니다. 이름으로 쓰면 실행 전에 SyntaxError가 납니다.

색으로 구분됩니다

VS Code에서는 키워드가 변수 이름과 다른 색으로 보입니다.

var이 기존의 방식입니다

```
var city = "서울";
var city = "부산";       // 같은 이름으로 다시 선언해도 오류가 없습니다
console.log(city);       // 부산

console.log(food);       // undefined ← 선언 전인데 오류가 아닙니다
var food = "김밥";

if (true) {
  var inside = "블록 안";
}
console.log(inside);     // 블록 안 ← { } 밖에서도 보입니다

/* 그래서 ES2015에서 let과 const가 나왔습니다 */
```

실수를 숨깁니다

같은 이름을 또 만들어도,
선언 전에 써도
오류 없이 넘어갑니다.

{ } 를 넘어갑니다

if 블록 안에서 만든 var가
밖에서도 보입니다.
var는 함수만 경계로 봅니다.

let은 다시 담을 수 있습니다

```
let age = 28;
age = 29;                // 다시 담기(재할당) : 됩니다
console.log(age);        // 29

let age = 30;            // 다시 선언(재선언) : 안 됩니다
// SyntaxError: Identifier 'age' has already been declared
```

재선언은 안 됩니다

같은 범위에서 let으로 같은
이름을 또 만들면 SyntaxError,
파일 전체가 멈춥니다.

선언 전에는 못 씁니다

선언하기 전에 쓰면
ReferenceError가 납니다.
var처럼 넘어가지 않습니다.

const는 다시 담을 수 없습니다

```
const birthYear = 1998;
birthYear = 2000;
// TypeError: Assignment to constant variable.

const country;
// SyntaxError: Missing initializer in const declaration

const user = { name: "김오즈" };
user.name = "이코딩";      // 됩니다 — 막는 것은 다시 담기뿐
```

선언할 때 값을 넣습니다

나중에 담을 수 없으니
선언하는 줄에서 바로
값을 넣어야 합니다.

바뀌지 않는 값에

태어난 해, 서비스 이름,
최대 인원처럼 한 번 정하면
그대로인 값에 씁니다.

var · let · const 한눈에

구분	var	let	const
다시 담기 (재할당)	됩니다	됩니다	TypeError
다시 선언 (재선언)	됩니다	SyntaxError	SyntaxError
선언 전에 쓰면	undefined	ReferenceError	ReferenceError
보이는 범위	함수	블록 { }	블록 { }
선언만 하기	됩니다 (undefined)	됩니다 (undefined)	SyntaxError
등장	처음부터	ES2015	ES2015

외울 것은 한 줄 — const는 다시 담을 수 없고, var는 새로 쓰지 않습니다

03 · 고르는 법

const를 먼저, 바뀔 값만 let

01 · 기본

const

다시 담을 일이 없으면
모두 const로 씁니다.
가장 먼저 고르는 선택지.

02 · 바뀔 때만

let

값이 바뀌는 것만 let입니다.
나이, 점수, 장바구니 개수처럼
다시 담는 값에 씁니다.

03 · 읽기만

var

새로 쓰지 않습니다.
남이 쓴 코드에서 만나면
읽을 수 있으면 됩니다.

MDN 코드 예제 작성 지침도 같은 순서입니다

기본은 const, 다시 담을 때만 let, var는 쓰지 않는다고 적혀 있습니다.

이름 짓기 — 지켜야 하는 규칙

```
let userName = "김오즈";    // 됩니다
let _count = 0;             // 됩니다 — 밑줄(_)로 시작
let $price = 12000;         // 됩니다 — 달러($)로 시작
let user2 = "이코딩";       // 됩니다 — 숫자는 두 번째부터

let 1stPlace = "금";       // SyntaxError — 숫자로 시작
let user-name = "김오즈"; // SyntaxError — - 는 빼기 기호
let user name = "김오즈"; // SyntaxError — 띄어 쓸 수 없음
let class = "A반";         // SyntaxError — 키워드
```

대소문자를 구분합니다

userName과 username은 다른 이름입니다. 오타 오류가 자주 생기는 곳입니다.

한글도 되지만

let 이름 = "김오즈" 도 실행됩니다. 다만 영어로 짓는 것이 약속입니다.

이름 짓기 — 지키면 좋은 약속

표기법	모양	쓰는 곳
camelCase (카멜)	`userName` `totalPrice`	변수와 함수 — JavaScript의 기본
PascalCase (파스칼)	`UserProfile`	클래스 이름 — 나중에 만납니다
UPPER_SNAKE_CASE	`MAX_COUNT`	처음부터 정해 두는 고정값
snake_case (스네이크)	`user_name`	Python 같은 다른 언어에서 흔합니다
kebab-case (케밥)	`user-name`	CSS 클래스와 파일 이름 — 변수에는 못 씀

이름만 보고 무엇이 들었는지 알 수 있게 짓습니다

a, data1 대신 userName처럼 짓고, 참·거짓을 담으면 isMember처럼 is·has로 시작합니다.

실습 01

콘솔과 파일에서 변수 다뤄 보기

Console에서 먼저 한 줄씩 쳐 보고, 같은 코드를 main.js로 옮겨 새로고침합니다.
다시 담기와 다시 선언이 어디서 막히는지 직접 확인합니다.

```
let city = "서울";
city = "부산";
console.log("도시 :", city);

const birthYear = 1998;
console.log("태어난 해 :", birthYear);
```

맨 아래에 넣어 보기

① birthYear = 2000;
→ TypeError, 위쪽 줄은 찍힘
② 지우고 let city = "대구";
→ SyntaxError, 한 줄도 없음

Console과 파일을 비교합니다

Console에서 친 let city는 다시 쳐도 오류가 나지 않습니다.

막히면

오류 줄 끝의 파일 이름을 누릅니다.

04

자료형과 디버깅

기본 자료형 · typeof · 오류 읽기 · 실습 02

값의 종류는 여덟 가지입니다

기본 자료형 (primitive)

일곱 가지

string 문자열 · number 숫자
boolean 참·거짓 · undefined · null
bigint 큰 정수 · symbol 고유한 값

객체 (object)

나머지 전부

배열, 객체, 함수처럼
여러 값을 묶어 담는 것이
모두 여기에 속합니다.

오늘은 앞의 다섯 개를 손에 익힙니다

bigint와 symbol은 이름만 알아 두고, 배열과 객체는 DAY 13에서 자세히 다룹니다.

문자열은 따옴표로 감쌉니다

```
let a = "큰따옴표";
let b = '작은따옴표';
let c = `백틱`;
console.log(typeof a, typeof b, typeof c);  // string string string

let d = "짝이 안 맞으면';
// SyntaxError: Invalid or unexpected token
```

셋 다 string

큰따옴표, 작은따옴표, 백틱 모두 문자열입니다. 한 파일 안에서는 하나로 맞춥니다.

여는 따옴표 = 닫는 따옴표

다른 따옴표로 닫으면 문자열이 끝나지 않아 SyntaxError가 납니다.

따옴표 안에 따옴표를 넣을 때

```
let s1 = '그는 "안녕"이라고 말했다';  // 다른 따옴표로 감쌉니다
let s2 = 'It\'s me';                  // \ 를 앞에 붙입니다
let s3 = "첫 줄\n둘째 줄";            // \n 은 줄바꿈
console.log(s3);
// 첫 줄
// 둘째 줄
```

역슬래시는 「글자 그대로」

따옴표 앞에 붙이면 문자열의
끝이 아니라 글자로 읽습니다.
이것을 이스케이프라고 합니다.

자주 쓰는 세 가지

`\n`	줄바꿈
`\'` `\"`	따옴표
`\\`	역슬래시

문자열을 잇는 두 가지 방법

```
const userName = "김오즈";
const age = 28;

// + 로 잇기
console.log("안녕하세요, " + userName + "님. " + age + "살입니다.");

// 템플릿 문자열 : 백틱 안에 ${ }
console.log(`안녕하세요, ${userName}님. ${age}살입니다.`);

// 둘 다 → 안녕하세요, 김오즈님. 28살입니다.
```

+ 는 빈칸도 직접

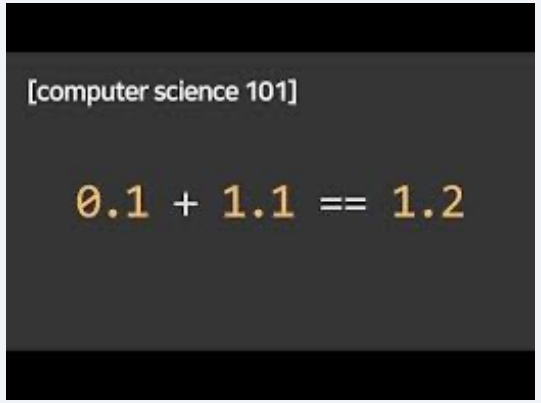
따옴표 안에 빈칸을 직접 넣어야 하고, 길어지면 읽기 어렵습니다.

백틱(`)은 1 왼쪽 키

${ } 안에 변수를 넣습니다. Mac 한글 입력이면 ₩ 가 찍힐 수 있어 영문으로 칩니다.

숫자는 정수와 소수를 나누지 않습니다

```
console.log(typeof 28);       // number
console.log(typeof 3.14);     // number ← 정수와 소수가 같은 종류
console.log(0.1 + 0.2);       // 0.30000000000000004
console.log(10 / 0);          // Infinity
console.log("사과" * 2);      // NaN (Not a Number)
console.log(typeof NaN);      // number
```



왜 그런지 영상으로

위 수식이 틀린 이유는?
(코딩애플, YouTube)
youtu.be/-GsrYvZoAdA

소수 계산의 오차

0.1을 2진수로 정확히
담지 못해 생기는 일입니다.
다른 언어도 같습니다.

금액은 원 단위 정수로

정수끼리 더하고 빼면
오차가 생기지 않습니다.
원 단위로 계산합니다.

참과 거짓은 boolean입니다

```
let isMember = true;
let hasCoupon = false;
console.log(typeof isMember);    // boolean

console.log(10 < 20);            // true  ← 비교한 결과도 boolean
console.log(10 > 20);            // false
console.log(typeof "true");      // string ← 따옴표가 있으면 문자열
```

값은 딱 두 개

true와 false뿐입니다.
따옴표 없이 씁니다.

조건문의 재료

DAY 11의 if는
이 참·거짓을 보고
갈 길을 정합니다.

비어 있음은 두 가지로 나타냅니다

undefined

아직 안 넣었습니다

선언만 하고 값을 넣지 않으면
JavaScript가 넣어 둡니다.
let nickname; → undefined

null

일부러 비웠습니다

값이 없다는 것을
내가 직접 표시합니다.
let phone = null;

typeof null은 "object"입니다

첫 구현 때부터 있던 오류입니다. 고치자는 제안이 있었지만 받아들여지지 않아 그대로입니다.

나머지 둘, bigint와 symbol

```
// 아주 큰 정수는 number로 정확히 담지 못합니다
console.log(9007199254740992 === 9007199254740993);    // true
// 끝에 n을 붙이면 bigint
console.log(9007199254740992n === 9007199254740993n);  // false
console.log(typeof 10n);                               // bigint

const id1 = Symbol("id");
const id2 = Symbol("id");
console.log(id1 === id2);   // false ← 설명이 같아도 서로 다른 값
console.log(typeof id1);    // symbol
```

bigint

9007199254740991
(Number.MAX_SAFE_INTEGER)
보다 큰 정수도 정확히
다룹니다.

symbol

만들 때마다 세상에
하나뿐인 값이 됩니다.
지금은 이름만 알아 둡니다.

typeof로 종류를 확인합니다

```
typeof "김오즈"      // "string"
typeof 28            // "number"
typeof true          // "boolean"
typeof undefined     // "undefined"
typeof null          // "object"  ← 앞에서 본 오래된 오류
typeof 10n           // "bigint"
typeof Symbol()      // "symbol"
```

결과도 문자열

typeof는 종류 이름을
문자열로 돌려줍니다.
typeof typeof 1은
"string"입니다.

괄호 없이 씁니다

typeof는 함수가 아니라
연산자라서
typeof age처럼 씁니다.

typeof가 헷갈리게 답하는 세 가지

01

typeof null

→ "object"
null인지 보려면
값 === null 로 비교합니다.

02

typeof []

→ "object"
배열도 객체라서 그렇습니다.
배열인지는 Array.isArray()로.

03

typeof NaN

→ "number"
계산에 실패해도
종류는 number로 남습니다.

typeof는 큰 갈래만 알려 줍니다 — 이상하면 값 자체를 찍어 봅니다

숫자처럼 보여도 문자열일 수 있습니다

```
const price = "12000";     // 입력칸에서 받은 값이라고 해 봅니다
const shipping = 3000;

console.log(price + shipping);           // 120003000 ← 이어 붙음
console.log(typeof price);               // string    ← 원인
console.log(Number(price) + shipping);   // 15000     ← 숫자로 바꿈
```

입력값은 문자열

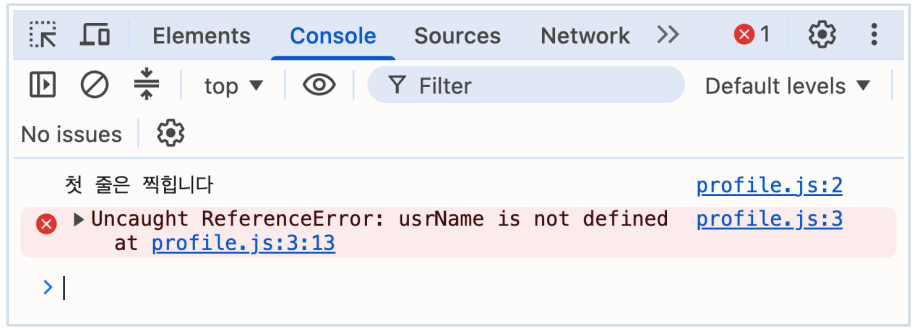
입력칸에 숫자를 적어도
JavaScript가 받는 값은
문자열입니다.

이상하면 typeof

결과가 이상할 때
typeof부터 찍으면
원인이 빨리 보입니다.

오류 메시지는 세 부분으로 읽습니다

```
const userName = "김오즈";
console.log("첫 줄은 찍힙니다");
console.log(usrName);    // 3번째 줄
```



01 · 종류

ReferenceError

어떤 종류의 문제인지.
이름을 찾지 못했다는
뜻입니다.

02 · 내용

usrName is not defined

무엇이 문제인지.
usrName이라는 이름이
어디에도 없다고 합니다.

03 · 위치

profile.js:3

몇 번째 줄인지.
누르면 Sources 탭의
그 줄로 이동합니다.

처음에 자주 만나는 오류 세 가지

SyntaxError

문법이 틀렸다

따옴표·괄호 짝, 숫자로 시작한 이름, 같은 이름 재선언. 파일이 한 줄도 안 돌아갑니다.

ReferenceError

없는 이름을 불렀다

오타, 대소문자 차이, 선언하기 전에 쓴 let·const. 그 줄에서 멈춥니다.

TypeError

할 수 없는 일을 시켰다

const에 다시 담기처럼 할 수 없는 일을 시킨 경우. 그 줄에서 멈춥니다.

SyntaxError는 실행 전에, 나머지 둘은 실행하다가 멈춥니다

그래서 SyntaxError가 있으면 맨 위의 console.log도 찍히지 않습니다.

console.log로 중간 값을 찍어 봅니다

```
let total = "0";                              // 실수 : 숫자 대신 문자열
total = total + 5000;
console.log("1단계", total, typeof total);  // 1단계 05000 string
total = total + 3000;
console.log("2단계", total, typeof total);  // 2단계 050003000 string

/* 1단계에서 이미 05000 — 첫 줄의 따옴표가 원인입니다 */
```

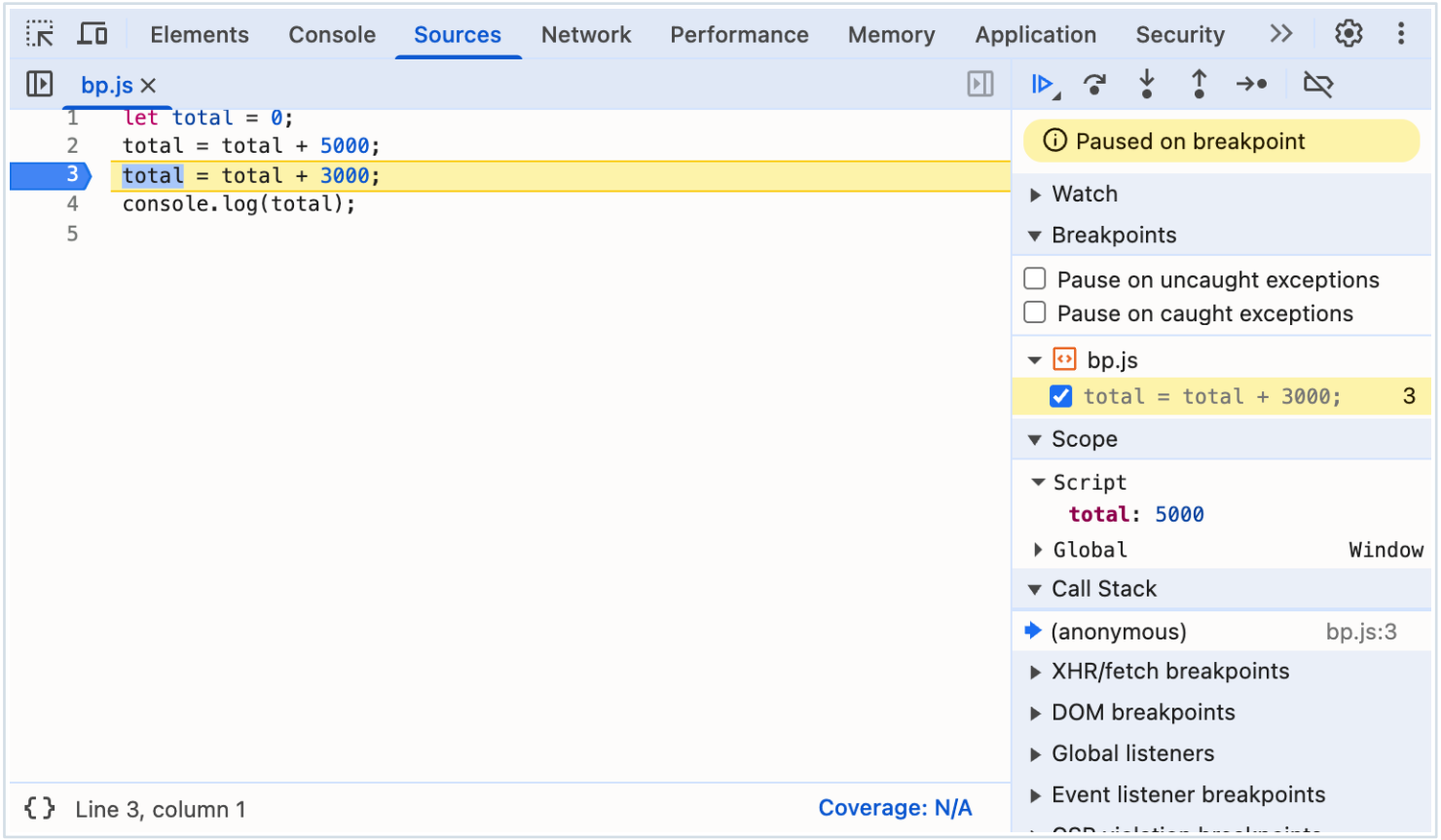
어디서 틀어졌나

단계마다 값을 찍으면
처음 어긋난 줄을
찾을 수 있습니다.

라벨을 붙여 찍기

"1단계"처럼 글자를 앞에 두면
여러 줄이 섞여도 구분됩니다.
다 고치면 지웁니다.

Sources 탭에서 멈춰 놓고 봅니다



3번 줄 번호를 눌러 멈춘 모습 — 오른쪽 Scope에 그 순간의 total 값 5000

멈춘 동안

변수에 마우스를 올리거나
오른쪽 Scope 칸에서
그 순간의 값을 봅니다.

한 줄씩 넘기기

F10은 다음 줄, F8은 계속 실행.
다 보면 줄 번호를 다시
눌러 중단점을 끕니다.

04 · 정리

막혔을 때 보는 순서

1	**Console 열기**	빨간 줄이 있는지 봅니다. 아무것도 없으면 파일 연결부터
2	**종류와 내용**	SyntaxError · ReferenceError · TypeError 중 무엇인지 읽기
3	**위치**	파일 이름:줄 번호를 눌러 그 줄로 이동
4	**글자 확인**	따옴표 짝, 대소문자, 괄호를 한 글자씩 비교
5	**값 찍기**	console.log와 typeof로 중간 값을 확인

그래도 모르면 오류 메시지를 그대로 복사해 검색합니다

실습 02

사용자 정보 출력하기

profile.js에 내 정보를 변수로 담고, 소개 문장과 자료형을 콘솔에 출력합니다.
바뀌지 않는 값은 const, 바뀌는 값은 let으로 고릅니다.

해야 할 것

이름과 이메일은 const로 담기

나이와 회원 여부는 let으로 담기

연락처는 null, 별명은 선언만 하기

템플릿 문자열로 소개 한 줄 출력

변수마다 typeof 결과 출력

확인할 것

소개 문장에 값이 제대로 들어간다

string · number · boolean이 나온다

phone은 object, nickname은 undefined

생일이 지나 age만 다시 담을 수 있다

Console에 빨간 줄이 하나도 없다

다 되면 커밋합니다 — 이어서 일부러 오류를 내 보고 고칩니다

실습 02

일부러 오류를 내고 고쳐 보기

완성한 profile.js를 한 곳씩 바꿔 저장하고, 오류 메시지를 읽은 뒤 되돌립니다.
한 번에 하나씩 바꿉니다. 되돌릴 때마다 새로고침해 빨간 줄이 사라졌는지 봅니다.

바꿀 곳	나오는 오류	찍히는 줄
① 이름 문자열의 닫는 " 를 ' 로	`SyntaxError: Invalid or unexpected token`	하나도 없음
② 소개 문장의 ${userName}을 ${usrName}으로	`ReferenceError: usrName is not defined`	그 줄 앞까지
③ 맨 아래에 userName = "이코딩"; 추가	`TypeError: Assignment to constant variable.`	그 줄 앞까지

세 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 02 끝

점검

JavaScript 첫날 자주 나오는 세 가지

!

Console이 비어 있어요

저장했는지, src 이름이 맞는지, 파일 어딘가에 SyntaxError가 없는지 봅니다.

!

is not defined가 떠요

이름 오타나 대소문자 차이입니다. userName과 username은 다릅니다.

!

constant variable이 떠요

const로 만든 변수에 다시 담았습니다. 바뀌는 값이면 let으로.

오류 메시지는 고칠 곳을 알려 주는 안내문입니다

종류, 내용, 파일 이름과 줄 번호를 차례로 읽으면 대부분 그 자리에서 풀립니다.

WRAP UP

오늘 정리

개념

JavaScript는 페이지에 동작을 더한다

브라우저의 엔진이 소스 코드를 바로 실행한다

const가 기본, 바뀔 값만 let, var는 안 쓴다

기본 자료형은 일곱, 나머지는 모두 객체다

typeof는 종류 이름을 문자열로 알려 준다

오류는 종류 · 내용 · 위치 순서로 읽는다

직접 쓸 수 있어야 하는 것

개발자 도구와 Console 열기

script 태그로 JS 파일 연결하기

let · const로 선언하고 값 바꾸기

템플릿 문자열로 값을 넣어 출력하기

typeof로 자료형 확인하기

오류 줄을 찾아가 고치기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

3장. 자바스크립트 문법

한빛미디어

『우아한 타입스크립트 with 리액트』

우아한형제들 웹프론트개발그룹 지음

2장. 타입

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

4장. 변수
5장. 표현식과 문
6장. 데이터 타입

NEXT

DAY 11

연산자와 조건문

오늘 담은 값으로 계산하고 비교합니다.
결과에 따라 다른 길을 가는 if와 switch를 배우고
점수 등급 계산기를 만듭니다.

수고하셨습니다.